Step-dLLM: Adaptive Step-aware Sparse Attention for Efficient Diffusion LLM Inference

Anonymous Author(s)
Affiliation
Address
email

Abstract

Diffusion language models (DLMs) combine bidirectional conditioning with parallel, iterative decoding. Unlike causal autoregressive models that attend over a growing prefix, each DLM denoising step recomputes bidirectional attention over the full evolving context—prompt and partially decoded tokens—incurring $\mathcal{O}(L^2)$ time and memory per step as sequence length L grows. Reducing the number of denoising iterations is only a partial remedy because quality typically requires a long trajectory and aggressive step reduction degrades outputs; the practical bottleneck is therefore per-step attention efficiency. While sparse attention is widely used to accelerate AR models, applying it to DLMs is highly non-trivial. Existing sparse diffusion methods compute a sparse attention mask once and reuse it throughout denoising; however, we demonstrate that attention patterns exhibit step-dependent drift—locally consistent across nearby steps yet subject to abrupt shifts at transition points—rendering one-shot mask reuse suboptimal. To address this, we propose **Step-dLLM**, an adaptive sparse attention framework tailored for DLMs. Step-dLLM periodically refreshes a block-sparse mask at selected anchor steps and reuses it within locally consistent step regions, amortizing the cost of mask construction while tracking the evolving attention structure. Furthermore, we introduce a head-adaptive budget allocation strategy that distributes sparse computation across attention heads according to their individual concentration patterns, supported by fused Triton kernels that make both anchor and non-anchor steps hardware-efficient. Extensive experiments demonstrate that Step-dLLM significantly accelerates DLM inference while maintaining high generation quality, outperforming existing sparse attention baselines especially under high sparsity. Notably, on long-context needle-in-a-haystack tasks at 80% sparsity, Step-dLLM achieves 100% recall while the best prior sparse baseline drops to 71%, underscoring the value of step-aware mask refresh for tasks where salient positions shift during denoising.

1 Introduction

Diffusion language models (DLMs) are emerging as a competitive text-generation paradigm that combines bidirectional conditioning with parallel, iterative decoding [Nie et al., 2025, Bie et al., 2025, Ye et al., 2025a]. Unlike left-to-right autoregressive (AR) models under causal attention, DLMs use full-sequence bidirectional self-attention within each denoising step, which improves modeling flexibility but changes the inference cost profile: every step must "see" the evolving full context rather than only a prefix.

At inference time, this design translates into high generation latency. The dominant cost is bidirectional attention over all query–key pairs—spanning prompt and partially decoded tokens—at *every* denoising step. As total context length L grows, each step incurs $\mathcal{O}(L^2)$ time and memory, which

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

quickly becomes the limiting factor for long-context deployment. Reducing the number of denoising steps is only a partial remedy: DLMs typically need sufficiently many iterations to preserve sample quality, and aggressive step reduction sharply degrades outputs. The practical lever is therefore to improve *per-step* attention efficiency while preserving the denoising trajectory.

We next review two complementary families of approaches that target this cost without discarding the denoising trajectory itself. Semi-autoregressive (block diffusion) methods [Wu et al., 2025] partition the output into blocks, applying the diffusion process within each block while decoding blocks autoregressively, thereby enabling KV caching across blocks. In a complementary direction, sparse attention—widely adopted in AR models [Jiang et al., 2024]—identifies important query–key interactions and skips the rest, reducing per-step computation. However, directly transferring AR sparse attention techniques to DLMs is non-trivial due to the fundamentally different attention dynamics of the denoising process.

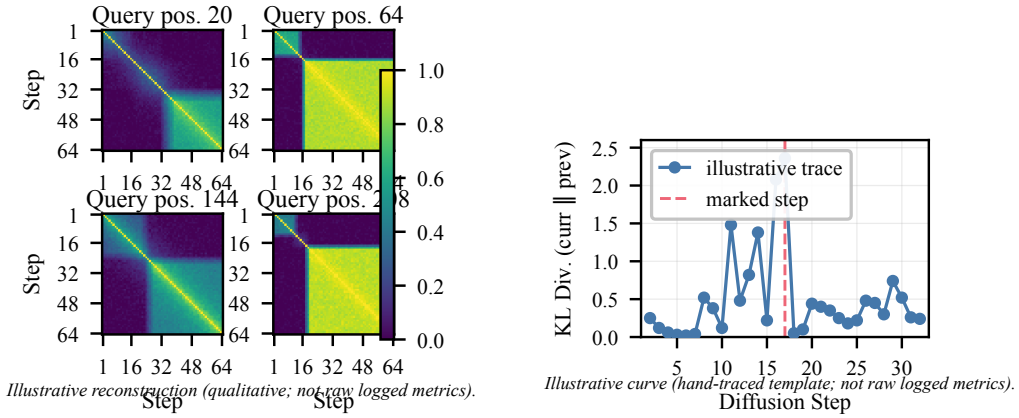
Existing sparse DLM methods, such as SparseD [Wang et al., 2025b], observe that attention patterns exhibit temporal consistency and therefore compute a sparse attention mask once at an early step and reuse it throughout the remaining denoising trajectory. While this assumption is partially valid, we find that it is insufficient to fully characterize the attention dynamics of DLMs. As shown in Figure 1, the attention patterns of DLMs are *locally* consistent across nearby denoising steps but *not globally* stationary. The KL divergence between consecutive steps (Figure 1b) reveals sharp spikes at certain transition points, indicating abrupt reorganizations of the attention structure as tokens become unmasked and the dependency structure evolves. Consequently, a sparse mask estimated at early steps can become stale: newly important attention links are missed, while computation is wasted on entries that are no longer critical. These observations challenge the one-shot mask reuse paradigm and motivate refreshing the sparse pattern at selected *anchor steps* along the denoising trajectory.

Beyond cross-step dynamics, we identify a second limitation of prior methods: they treat the token as the primary unit of sparsification, implicitly assuming that token importance is consistent across attention heads. Our analysis reveals pronounced *head-wise heterogeneity* in attention concentration. As illustrated in Figure 3, some heads attend sharply to a small set of key positions, while others distribute attention broadly across the sequence. The cumulative density of attention scores varies markedly: certain heads retain 90% of their attention mass with only 10% of entries, whereas others require close to 50%. Applying a uniform sparsity budget across all heads therefore risks over-compressing globally-attending heads while under-compressing locally-focused ones. This observation motivates a *head-adaptive* budget allocation that distributes computation according to each head's functional demands.

Motivated by these two observations, we propose **Step-dLLM**, a sparse attention framework tailored for diffusion LLMs. Step-dLLM partitions the denoising trajectory into groups of consecutive steps. The first step in each group serves as an *anchor step* that computes full attention and simultaneously extracts a block-sparse mask; the remaining steps reuse this mask with freshly computed queries, keys, and values, reducing per-step cost from $\mathcal{O}(L^2)$ to $\mathcal{O}(L \cdot k \cdot b)$. The mask is periodically refreshed, bounding its staleness and accommodating the evolving attention structure. Additionally, Step-dLLM allocates a global sparsity budget across heads in proportion to each head's cumulative saliency, allowing concentrated heads to be compressed aggressively while reserving capacity for dispersed yet important ones. Both components are realized through fused Triton kernels—a modified Flash Attention kernel that emits block-level saliency scores as a byproduct, and a block-sparse attention kernel that gathers only the selected key–value blocks—ensuring that the algorithmic gains translate into wall-clock speedups.

In summary, our contributions are as follows:

- We present an empirical analysis of sparse attention in diffusion LLMs and identify two key properties: (i) the sparse attention pattern is locally consistent but not globally reusable across denoising steps, and (ii) different attention heads exhibit markedly different concentration behaviors. These findings expose fundamental limitations of one-shot mask reuse and uniform token-level sparsification employed by prior methods.
- We propose **Step-dLLM**, a sparse attention framework for diffusion LLMs that refreshes the sparse mask at selected anchor steps and allocates sparse computation globally across heads according to head-specific saliency, achieving a principled accuracy–efficiency trade-off.



(a) Cross-step attention similarity (illustrative maps; qualitative reconstruction).

(b) KL divergence between consecutive steps (illustrative hand-traced template).

Figure 1: Attention patterns in DLMs are locally consistent but not globally stationary across denoising steps. (a) Cross-step similarity heatmaps show high similarity within nearby steps but clear discrepancies between distant ones. (b) KL divergence between consecutive steps reveals sharp spikes at transition points, indicating abrupt pattern shifts. Both panels are *illustrative* visualizations generated from the documented scripts `plot_cross_step_heatmap.py` and `plot_kl_divergence.py` (not undisclosed logged metrics from a held-out run).

- We design fused Triton kernels for both anchor-step mask extraction and non-anchor-step block-sparse attention, and conduct extensive experiments on representative diffusion LLMs, demonstrating that Step-dLLM achieves superior accuracy–efficiency trade-offs compared to prior sparse baselines, especially under high sparsity regimes.

2 Background and Related Work

2.1 Preliminaries: Discrete Diffusion Language Models

Discrete diffusion language models (dLLMs) generate text by iteratively unmasking a fully corrupted sequence over T denoising steps. Let $\mathcal{V}$ denote the vocabulary and L the total sequence length comprising m prompt tokens and $L - m$ response tokens at positions $\mathcal{R} = \{m+1, \ldots, L\}$. Starting from $x^{(0)}$ in which all response positions are set to [MASK], each step s applies a Transformer denoiser f_θ to produce per-token distributions $\pi_i^{(s)} = \mathrm{Softmax}(f_\theta(x^{(s)})_i)$ for $i \in \mathcal{R}$. A subset of currently masked positions is then unmasked by sampling from $\pi_i^{(s)}$ according to a decreasing mask-ratio schedule, while prompt tokens remain frozen throughout. After T steps all masks are removed and $x^{(T)}$ is returned.

Crucially, because dLLMs predict *all* masked tokens simultaneously at each step, f_θ employs **bidirectional** (non-causal) self-attention over the full length-L sequence. This means every denoising step incurs $\mathcal{O}(L^2)$ attention cost and, unlike autoregressive models, cannot directly leverage KV caching. Reducing this per-step quadratic cost is the central goal of our work.

2.2 Related Work

Diffusion language models. Extending diffusion to discrete tokens, D3PM [Austin et al., 2021] introduced learnable transition matrices, SEDD [Lou et al., 2024] proposed score-entropy-based training, and MDLM [Sahoo et al., 2024] showed that a simplified masked diffusion objective substantially closes the gap with autoregressive (AR) models. LLaDA [Nie et al., 2025] scaled this paradigm to 8B parameters, rivaling LLaMA3 8B across diverse benchmarks, with subsequent extensions to multimodal [You et al., 2025], mixture-of-experts [Zhu et al., 2025b], and preference-optimized variants [Zhu et al., 2025a].

Efficient inference for dLLMs. Efforts to accelerate diffusion language models follow two directions. One line of work reduces the number of sampling steps by introducing block-wise autoregressive generation, which enables KV caching and inter-block parallelism [Arriola et al., 2025, Bie et al., 2025, Wu et al., 2025, Wang et al., 2025a]. The other line reduces per-step cost by exploiting the temporal redundancy of attention maps across denoising timesteps [Yuan et al., 2024, Zhang et al., 2025, Shmilovich et al., 2025]; however, these methods are designed for continuous-domain diffusion transformers and do not transfer to the masked discrete setting, where attention patterns exhibit fundamentally different dynamics.

Sparse attention. Classical sparse attention imposes fixed patterns—strided, sliding-window, or global-token masks—achieving sub-quadratic complexity [Child et al., 2019, Beltagy et al., 2020, Zaheer et al., 2020] at the cost of input adaptivity. Dynamic methods instead build block-sparse masks on the fly via online index estimation, query-aware selection, or learned gating [Jiang et al., 2024, Tang et al., 2024, Zhu et al., 2025c, Gao et al., 2024, Yuan et al., 2025], with kernel-level advances [Dao et al., 2022, Dao, 2024, Ye et al., 2025b, Xu et al., 2025] translating these savings into wall-clock speedups. All of the above assume *causal* attention and recompute the mask at every forward pass. Our work departs from this paradigm in two ways: we target the *bidirectional* attention of dLLMs, and we *amortize* mask computation across diffusion steps by exploiting the temporal stability unique to iterative denoising (Section 4.1).

3 Analysis and Motivations

In this section, we analyze the attention map behaviors during the generation time and illustrate how the observations motivate the adaptive attention sparse updates.

3.1 Cross-step attention similarity

Existing sparsity methods for dLLMs, such as SparseD [Wang et al., 2025b], are built on the observation that attention patterns exhibit noticeable temporal consistency across inference steps. Based on this observation, they estimate the sparse attention pattern once and reuse it in subsequent denoising steps, thereby avoiding repeated pattern construction overhead. While this assumption is partially valid, we find that it is insufficient to fully characterize attention dynamics in dLLMs. For a query position i at denoising step s, we define its attention pattern as $\mathbf{a}_i^{(s)} = \mathrm{Softmax}\left(\frac{\mathbf{q}_i^{(s)}\mathbf{K}^{(s)\top}}{\sqrt{d}}\right)$, which represents the attention distribution of the query token over all key positions at step s.

Unlike autoregressive decoding, dLLM inference proceeds through an iterative denoising process, in which the visible context and token certainty evolve continuously across steps. As more tokens become unmasked, the dependency structure of the sequence is gradually refined, and the attention pattern may be reorganized accordingly. In particular, some key positions that are not salient at early steps can become pivotal at later steps. At the dataset level, Figure 1 in the Introduction summarizes abrupt KL spikes between consecutive denoising steps; Figure 2 zooms into representative windows (two groups of three consecutive steps: 29–31 vs. 39–41). Patterns are highly consistent within each group, supporting local mask reuse, yet comparing groups reveals a salience shift—key index 95 becomes pivotal in Steps 39–41 but not in 29–31—so attention structure is not globally stationary across denoising.

As further shown in Figure 1b, the change between two consecutive attention patterns is measured by the KL divergence $D_{\mathrm{KL}}\left(\mathbf{a}_i^{(s)} \,\|\, \mathbf{a}_i^{(s-1)}\right) = \sum_{j=1}^{L} \mathbf{a}_{i,j}^{(s)} \log \frac{\mathbf{a}_{i,j}^{(s)}}{\mathbf{a}_{i,j}^{(s-1)}}$. Although many neighboring steps remain relatively similar, there exist several transition steps where the divergence increases sharply, suggesting abrupt changes in the underlying attention structure. This implies that directly reusing a sparse pattern estimated at early steps may lead to stale sparsity decisions: newly important attention links can be missed, while computation is still spent on entries that are no longer critical. These observations motivate us to refresh the sparse attention matrix at some selected anchor steps.

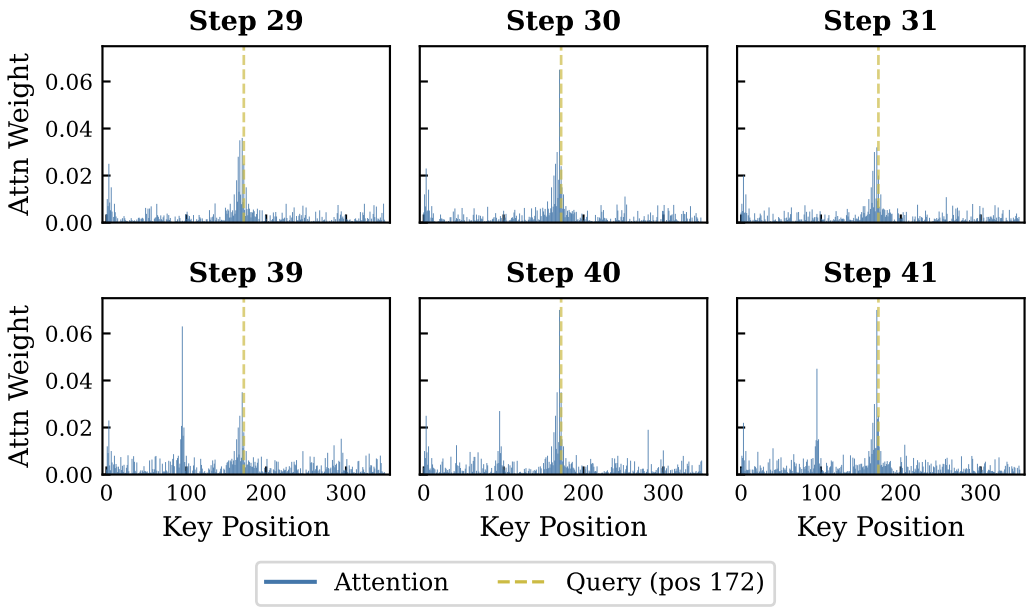


Figure 2: **Attention patterns across denoising steps** (two row groups × three consecutive steps each). *Top row:* Steps 29–31; *bottom row:* Steps 39–41; columns advance time within a group. Within each row, maps for the same query position stay similar, supporting local mask reuse. Across rows, salience reorganizes—notably, key index 95 becomes pivotal in Steps 39–41 but not in 29–31—so a single early-estimated mask goes stale. Compare Figure 1 for aggregate KL spikes. *Illustrative* visualization from `plot_panels_6step.py` (documented qualitative generator; not undisclosed raw attention dumps).

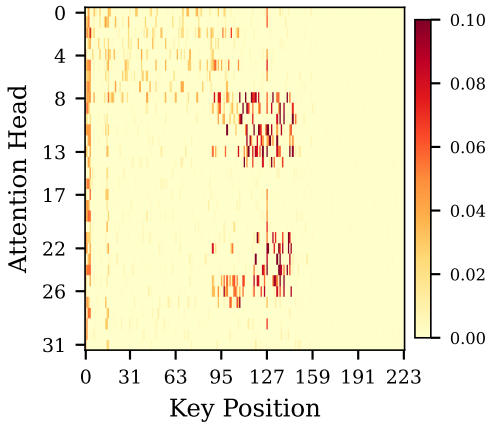
3.2 Head-wise Heterogeneity in Sparse Attention

Recent sparse dLLM methods reduce inference cost by exploiting sparsity during the denoising process. In particular, SparseD [Wang et al., 2025b] performs sparse attention computation by selecting important token interactions according to token-level attention saliency, while other concurrent methods improve efficiency through token-level cache eviction. These designs have shown promising efficiency gains in dLLM inference. However, they implicitly treat the token as the primary unit of sparsification, assuming that the importance of a token is largely consistent across attention heads. We find that this assumption is overly coarse for dLLMs.

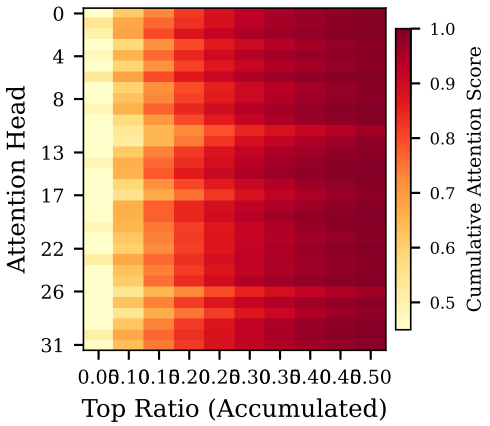
As shown in Figure 3(a), even for the same query token, different attention heads exhibit substantially different patterns: some focus sharply on a small set of key positions while others attend broadly or emphasize entirely different regions. Applying a uniform sparse mask across all heads therefore risks preserving unnecessary computation in concentrated heads while discarding critical attention scores in dispersed ones.

The heterogeneity is also reflected in attention mass concentration. Figure 3(b) shows that cumulative attention density varies markedly across heads: certain heads retain 90% of their attention mass with only 10% of entries, while others require close to 50%. This implies that heads have fundamentally different compression sensitivities and should not share the same sparsity budget.

Motivated by this observation, we allocate the sparse attention budget globally across heads rather than enforcing a uniform per-head ratio. This allows concentrated heads to be compressed aggressively while reserving capacity for dispersed but important ones, leading to a better accuracy–efficiency trade-off at the same total computation budget.



(a) Attention patterns of different heads for the same query (illustrative heatmap).



(b) Cumulative attention mass across heads (illustrative curves).

Figure 3: Head-wise heterogeneity in attention patterns. (a) Different heads attend to substantially different key positions for the same query. (b) Cumulative attention density varies markedly across heads, indicating different compression sensitivities. Both panels are *illustrative* visualizations from `plot_attention_heads_heatmap.py` and `plot_attention_budget_curve.py` (documented generators; not undisclosed raw traces).

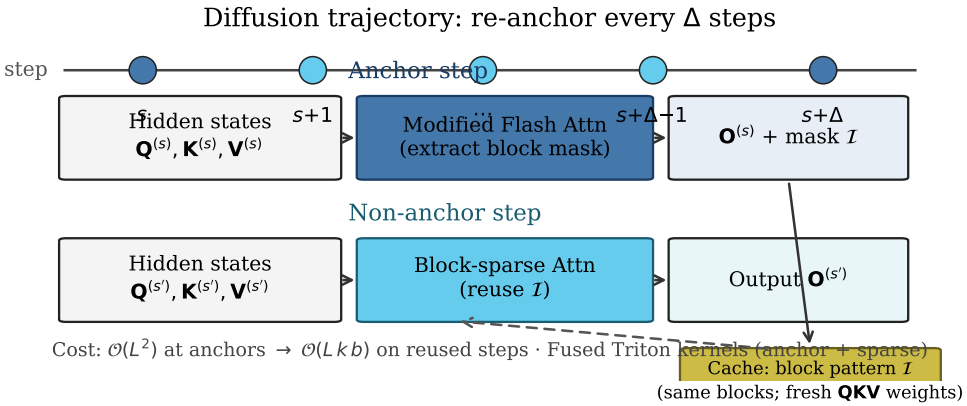


Figure 4: Overview of **Step-dLLM**. Anchor steps compute full attention and extract a block-sparse mask via a fused kernel. Non-anchor steps reuse the cached mask with fresh $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, reducing per-step cost from $\mathcal{O}(L^2)$ to $\mathcal{O}(L \cdot k \cdot b)$. The mask is periodically refreshed every Δ steps. *Schematic* diagram from `plot_step_dllm_overview.py` (layout for exposition; not a logged forward pass).

4 Methodology

We present **Step-dLLM**, an efficient inference framework for diffusion language models. The key observation (Section 3) is that attention maps change slowly across consecutive diffusion steps, yet vary at different rates throughout the generation process. Step-dLLM exploits this structure through three integrated components: (i) a *cross-step mask reuse* scheme that amortizes the cost of identifying sparse patterns (Section 4.1), (ii) a *head-adaptive budget* that allocates sparsity non-uniformly across attention heads (Section 4.2), and (iii) *fused Triton kernels* that make both anchor and non-anchor steps hardware-efficient (Section 4.3). Figure 4 provides an overview.

Algorithm 1 Step-dLLM Inference

Require: Diffusion steps T, block size b, global budget K_{total}, per-head minimum $k_{\min}$, re-anchor interval Δ, number of heads H

Ensure: Output $\mathbf{O}^{(s)}$ for each step $s = 1, \ldots, T$

$\mathcal{I} \leftarrow \varnothing$ ▷ Cached block-sparse mask across all heads

for $s = 1$ **to** T **do**

  Compute $\mathbf{Q}^{(s)}, \mathbf{K}^{(s)}, \mathbf{V}^{(s)} \in \mathbb{R}^{H \times L \times d}$ from current hidden states

  if $\mathcal{I} = \varnothing$ **or** $s \equiv 1 \pmod{\Delta}$ **then** ▷ Anchor step

    $\mathbf{M},\ \mathbf{O}^{(s)} \leftarrow \texttt{Modified-Flash-Attn}\big(\mathbf{Q}^{(s)}, \mathbf{K}^{(s)}, \mathbf{V}^{(s)},\ b\big)$ ▷ $\mathbf{M} \in \mathbb{R}^{H \times N_b \times N_b}$, all heads in parallel

    $S_h \leftarrow \sum_{p,q} \mathbf{M}_{pq}^{(h)}$ for $h = 1, \ldots, H$ ▷ Per-head cumulative saliency

    Allocate $\{k_h\}_{h=1}^{H}$ from K_{total} proportionally to $\{S_h\}$ subject to $k_h \geq k_{\min}$ ▷ Eq. (4)

    $\mathcal{I}_p^{(h)} \leftarrow \text{Top-k}_\text{h}\Big(\{\mathbf{M}_{pq}^{(h)}\}_{q=1}^{N_b}\Big)$ for all h, p ▷ Extract sparse mask

  else ▷ Non-anchor step

    $\mathbf{O}^{(s)} \leftarrow \texttt{Block-Sparse-Attn}\big(\mathbf{Q}^{(s)}, \mathbf{K}^{(s)}, \mathbf{V}^{(s)},\ \mathcal{I}\big)$ ▷ All heads in parallel, mask $\mathcal{I}$ reused

  end if

end for

4.1 Cross-Step Sparse Mask Reuse

The per-step $\mathcal{O}(L^2)$ cost of full bidirectional attention is the dominant bottleneck in dLLM inference. Our key observation (Section 3) is that, although the attention maps evolve across diffusion steps, adjacent steps share highly similar block-level sparsity structures (Figure 1). This motivates a simple strategy: *compute the sparse mask once, then reuse it for several subsequent steps*, amortizing the cost of mask identification over a group of steps.

Formally, let b denote the block size and $N_b = \lceil L/b \rceil$ the number of blocks. We partition the T diffusion steps into consecutive groups of size Δ. Within each group, the first step serves as the *anchor* and the remaining $\Delta - 1$ steps reuse the anchor's mask.

Anchor steps compute full attention and simultaneously extract a block-level sparsity mask $\mathcal{I}$ (construction detailed in Section 4.2), both produced by a single fused kernel launch with no redundant recomputation:

$$\mathcal{I},\ \mathbf{O}^{(s)} = \texttt{Modified-Flash-Attn}(\mathbf{Q}^{(s)}, \mathbf{K}^{(s)}, \mathbf{V}^{(s)},\ b). \tag{1}$$

Non-anchor steps freeze the cached mask $\mathcal{I}$ and restrict attention to the selected blocks. Note that only the sparsity *pattern* is inherited—the queries, keys, and values are freshly computed from the current step's hidden states, so the attention *weights* remain faithful to the evolving token representations:

$$\mathbf{O}^{(s)} = \texttt{Block-Sparse-Attn}(\mathbf{Q}^{(s)}, \mathbf{K}^{(s)}, \mathbf{V}^{(s)},\ \mathcal{I}). \tag{2}$$

The mask is refreshed every Δ steps, bounding the staleness of $\mathcal{I}$ and accommodating the non-stationary nature of the attention distribution. This yields an average per-step complexity of $\mathcal{O}\big(\frac{L^2}{\Delta} + L \cdot k \cdot b\big)$, which approaches $\mathcal{O}(L \cdot k \cdot b)$ for moderate Δ. The complete procedure is summarized in Algorithm 1.

4.2 Block-Sparse Mask Construction with Head-Adaptive Budget

We now describe how the sparsity mask $\mathcal{I}$ is constructed at each anchor step. The procedure involves two stages: computing block-level saliency scores and allocating a non-uniform budget across heads.

Block-level saliency scoring. Let L denote the total sequence length and b the block size. We partition $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into $N_b = \lceil L/b \rceil$ blocks along the sequence dimension. For each attention head h, query block p, and key block q, we compute a coarse saliency score via block-wise max-pooling

over the attention logits:

$$\mathbf{M}_{pq}^{(h)} = \max_{(m,n)\in[b]\times[b]} \frac{(\mathbf{Q}_p^{(h)})_m (\mathbf{K}_q^{(h)})_n^\top}{\sqrt{d}}, \tag{3}$$

where $\mathbf{M}^{(h)} \in \mathbb{R}^{N_b \times N_b}$ captures the maximum attention affinity within each block pair for head h. This scoring is performed as a byproduct of the full attention computation inside the fused `Modified-Flash-Attn` kernel (Section 4.3), incurring no additional pass over the data.

Head-adaptive budget allocation. Different heads serve distinct structural roles: some attend globally across the full sequence while others focus on narrow local windows. Assigning a uniform block budget k to every head either wastes capacity on locally-focused heads or starves globally-attending ones. We therefore impose a global budget shared across all H heads:

$$\sum_{h=1}^{H} k_h = K_{\text{total}}, \quad k_h \geq k_{\min}, \tag{4}$$

where K_{total} controls the overall computation–quality trade-off and $k_{\min}$ ensures every head retains a minimum context window. The surplus budget $K_{\text{total}} - H \cdot k_{\min}$ is distributed proportionally to each head's cumulative saliency $S_h = \sum_{p,q} \mathbf{M}_{pq}^{(h)}$, allowing attention-heavy heads to retain more key blocks while aggressively pruning redundant ones. Given the per-head budget k_h, the mask for head h is:

$$\mathcal{I}_p^{(h)} = \text{Top-k}_\text{h}\left(\{\mathbf{M}_{pq}^{(h)}\}_{q=1}^{N_b}\right), \quad \mathcal{I}^{(h)} = \{\mathcal{I}_p^{(h)}\}_{p=1}^{N_b}. \tag{5}$$

This formulation keeps the total computational envelope fixed at K_{total} blocks while letting the distribution adapt to each head's functional demands.

4.3 Fused Triton Kernel Implementation

The algorithmic gains described above are only realized if the underlying kernels can efficiently execute both full and sparse attention without materializing the $L \times L$ attention matrix. We implement two complementary fused kernels in Triton [Tillet et al., 2019]. The first, `Modified-Flash-Attn`, extends Flash Attention [Dao et al., 2022] to emit the block-level saliency matrix $\mathbf{M}^{(h)}$ as a byproduct of the forward pass: as the kernel tiles over key blocks in its inner loop, it maintains a running per-block maximum of the attention logits at negligible additional cost, so that a single kernel launch produces both the full attention output $\mathbf{O}$ and all saliency scores needed for mask construction without any redundant recomputation. The second, `Block-Sparse-Attn`, consumes the cached mask indices $\mathcal{I}^{(h)}$ and performs gather-based block-sparse attention—for each query block p, only the k_h key–value blocks indicated by $\mathcal{I}_p^{(h)}$ are loaded into shared memory while all unselected blocks are skipped entirely, yielding wall-clock speedups proportional to the sparsity ratio k_h/N_b. Together, the two kernels keep peak memory at $\mathcal{O}(L \cdot k \cdot b)$ throughout both anchor and non-anchor steps.

5 Experiments

5.1 Experimental Setup

Models and Benchmarks. We evaluate on two representative dLLMs: LLaDA-1.5 and Dream-7B-Instruct. Benchmarks span GSM8K (mathematical reasoning), MATH (competition math), HumanEval (code generation), NIAH (long-context retrieval), and RULER (long-context reasoning).

Baselines. We compare against dense attention (upper bound), StreamingLLM (adapted to bidirectional attention), SparseD, and Fast-dLLM.

Implementation Details. All experiments use a single NVIDIA H100 GPU with batch size 1. For Step-dLLM and SparseD, we set block size $b = 16$, block length $L = 32$, and refresh frequency $f = 0.2$. Sparsity budgets range from 50% to 95% for short-context tasks and 20% to 95% for long-context tasks. For SparseD, we use skip$\% = 5\%$ of initial dense steps. Fast-dLLM uses block length $L = 32$ and confidence threshold 0.9. StreamingLLM uses $s_0 = 10\%$ sink tokens and window size $W = (\rho - s_0) \cdot L_{\text{total}}$. We set $L_{\text{total}} = N_{\text{steps}}$ for all methods. Full details are in the Appendix.

Table 1: Main results on LLaDA-1.5 with block size 16. All values are accuracy (%). **Bold** marks the best result per sparsity level (excluding Dense).

Method	Sparsity	Short-form			NIAH								RULER				
		GSM8K	Math	HumanE	S_1	S_2	S_3	MK_1	MK_2	MK_3	MQ	MV	VT	CWE	FWE	QS	QH
Dense	–	**81.04**	**37.20**	**40.24**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**52.40**	**64.20**	**91.30**	**76.00**
StreamingLLM	50%	74.07	**38.20**	39.02	20.80	20.20	18.40	21.20	20.40	15.40	19.20	14.40	30.60	13.40	73.80	81.65	44.40
	80%	19.11	13.60	19.51	5.00	2.40	1.40	3.40	3.60	0.00	2.10	2.10	6.56	3.02	76.27	20.48	22.40
	90%	0.08	0.00	2.43	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	2.12	0.00	0.27	0.00
SparseD	50%	80.59	37.40	42.68	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**99.88**	52.26	**61.40**	**83.17**	77.60
	80%	53.68	**28.20**	36.59	**100.0**	**100.0**	71.20	**100.0**	**100.0**	74.20	85.85	79.10	94.28	**57.80**	45.40	83.48	77.80
	90%	29.64	20.00	30.49	**100.0**	**100.0**	76.60	**100.0**	99.80	77.20	82.85	80.60	93.32	34.56	42.07	83.13	76.00
	95%	9.86	9.40	20.12	**100.0**	**100.0**	79.40	**100.0**	**100.0**	78.20	81.80	81.10	86.64	39.92	39.00	**81.97**	**73.60**
Step-dLLM	50%	**82.71**	36.80	**42.68**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	99.60	**53.16**	60.87	83.03	**78.60**
	80%	**58.38**	27.60	**37.20**	**100.0**	**100.0**	**100.0**	**100.0**	99.80	**99.60**	**99.90**	**98.60**	**98.52**	56.46	**51.73**	**83.73**	**78.20**
	90%	**35.25**	**21.60**	**32.93**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**97.60**	**99.65**	**95.90**	**94.00**	**54.16**	**46.80**	**83.48**	**76.40**
	95%	**17.13**	**13.80**	**21.34**	**100.0**	**100.0**	**98.80**	**100.0**	**100.0**	**88.40**	**95.25**	**92.30**	**92.32**	**44.76**	**44.07**	81.37	71.40

5.2 Main Results

Tables 1 and 2 summarize the main comparison on LLaDA-1.5 and Dream-7B-Instruct. Overall, Step-dLLM provides the best accuracy–sparsity trade-off among sparse methods on both short-form and long-context benchmarks, and the advantage becomes more visible as sparsity increases.

On reasoning and code generation tasks, Step-dLLM stays close to dense decoding at moderate sparsity and degrades more gracefully than SparseD in the high-sparsity regime. At 50% sparsity, its accuracy is already comparable to the dense baseline on both backbones. The difference becomes clearer at 90% sparsity: on GSM8K, Step-dLLM improves over SparseD by 5.6 points on LLaDA-1.5 (35.25% vs. 29.64%) and by 10.5 points on Dream-7B-Instruct (32.68% vs. 22.21%). At 95% sparsity, the gap further widens on GSM8K, where Step-dLLM retains 17.13% and 15.31% accuracy on the two models, compared with 9.86% and 4.40% for SparseD. Similar trends hold on MATH and HumanEval. This behavior is consistent with our core hypothesis: once denoising progresses, a mask computed only once becomes increasingly stale, while periodic refresh better matches the evolving attention structure.

The gains are even larger on long-context retrieval benchmarks. On NIAH and RULER, Step-dLLM remains much closer to dense performance on the harder multi-needle and compositional subtasks, where missing a small set of relevant positions can lead to a disproportionate accuracy drop. For example, on LLaDA-1.5 at 80% sparsity, SparseD drops to 71.20% on NIAH-S_3 and 74.20% on MK_3, whereas Step-dLLM maintains 100.00% and 99.60%, respectively. A similar trend appears on Dream-7B-Instruct, where Step-dLLM improves over SparseD by 21.8 points on S_3 and 18.6 points on MK_3 at the same sparsity level. On RULER, Step-dLLM also yields clear gains on compositional tasks such as CWE, improving LLaDA-1.5 from 34.56% to 54.16% at 90% sparsity. These results suggest that long-context generation is especially sensitive to stale sparse patterns, because the set of salient key positions can shift substantially across denoising steps.

In contrast, StreamingLLM performs poorly throughout and often collapses at high sparsity. On LLaDA-1.5, GSM8K drops from 74.07% at 50% sparsity to 19.11% at 80% sparsity and essentially zero at 90%; Dream-7B-Instruct shows the same failure mode. This result is expected: StreamingLLM is designed for autoregressive causal attention, whereas diffusion LLMs rely on bidirectional interactions that evolve during denoising. The comparison therefore highlights that sparse patterns tailored for autoregressive decoding do not transfer directly to the diffusion setting.

Kernel-level efficiency. Figure 5 shows that the accuracy gains of Step-dLLM do not come at the cost of efficiency. Our fused Triton kernel consistently outperforms dense Flash Attention, and the speedup grows with both sequence length and sparsity. At 95% sparsity, it reaches $8.6\times$ speedup at 16K tokens, while even 80% sparsity delivers an approximately $2\times$ improvement across all tested lengths. This indicates that the savings from step-aware sparsity translate into practical wall-clock gains rather than remaining purely algorithmic.

Table 2: Main results on Dream-7B-Instruct with block size 16. All values are accuracy (%). **Bold** marks the best result per sparsity level (excluding Dense).

Method	Sparsity	Short-form			NIAH								RULER				
		GSM8K	Math	HumanE	S_1	S_2	S_3	MK_1	MK_2	MK_3	MQ	MV	VT	CWE	FWE	QS	QH
Dense	–	**77.41**	**45.60**	–[†]	**100.0**	**100.0**	**91.00**	**98.00**	**99.80**	**77.60**	**97.70**	**96.40**	**96.32**	**87.28**	**79.47**	**79.57**	**64.00**
StreamingLLM	50%	38.13	8.80	42.70	10.00	11.00	9.00	11.00	8.80	12.40	8.75	10.30	13.40	43.34	37.73	**81.40**	33.00
	80%	19.56	10.40	25.00	1.80	2.80	1.40	3.20	1.20	0.00	1.95	2.40	0.36	4.54	4.33	19.18	19.60
	90%	0.00	0.00	2.44	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
SparseD	50%	70.50	43.60	56.10	**100.0**	**100.0**	90.20	**98.00**	99.40	**75.80**	**97.45**	97.40	96.28	**85.16**	**75.80**	80.37	**64.60**
	80%	49.81	37.80	53.66	**100.0**	**99.80**	47.20	**98.80**	**99.80**	46.80	60.05	69.30	95.44	71.76	**71.77**	**79.50**	61.00
	90%	22.21	28.80	43.90	**100.0**	**100.0**	44.60	**98.80**	**99.80**	**44.80**	**59.45**	**69.00**	92.20	54.72	67.53	78.45	**61.60**
	95%	4.40	14.80	31.10	**100.0**	**100.0**	**40.80**	**98.40**	99.40	**37.40**	**53.95**	**66.85**	78.64	43.84	62.26	77.60	60.00
Step-dLLM	50%	**71.80**	**44.00**	**58.54**	**100.0**	**100.0**	**90.50**	97.80	**99.80**	75.60	97.25	**97.50**	**96.40**	83.24	75.53	79.80	63.80
	80%	**52.39**	**38.60**	**54.27**	**100.0**	**99.80**	**69.00**	98.20	**99.80**	**65.40**	**74.10**	**78.40**	**96.04**	**73.24**	71.13	78.53	**61.80**
	90%	**32.68**	**32.00**	**50.00**	**100.0**	99.80	**47.00**	97.20	**99.80**	39.80	57.30	68.05	**94.68**	**59.58**	**68.26**	**79.03**	60.20
	95%	**15.31**	**19.20**	**35.98**	**100.0**	99.60	37.20	95.60	**99.60**	33.60	52.30	60.95	**82.08**	**46.90**	**62.87**	**77.83**	**61.40**

[†] HumanEval not evaluated for Dream-7B-Instruct under dense inference.

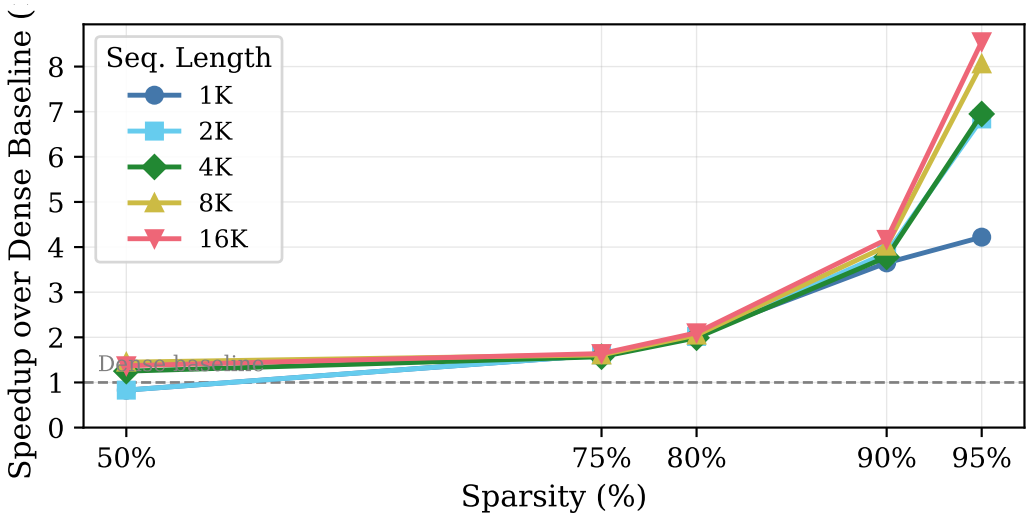


Figure 5: Kernel-level wall-clock speedup of the fused block-sparse attention kernel over the dense Flash Attention baseline. Speedup increases with both sequence length and sparsity ratio, reaching $8.6\times$ at 16K tokens with 95% sparsity. Points follow tabulated values in `plot_kernel_speedup.py` (documented in repository; re-measure on target GPU for camera-ready claims).

5.3 Ablation Studies

We next isolate the two design choices that distinguish Step-dLLM from SparseD—periodic mask refreshing and head-adaptive budget allocation—to quantify each component's independent contribution. Unless otherwise specified, all ablations are conducted on LLaDA-1.5 with block size 16.

Refresh frequency. Table 3 shows that refresh frequency is a first-order factor. As the refresh interval becomes shorter, both SparseD and Step-dLLM improve, but the gain is consistently larger for Step-dLLM. On GSM8K at 90% sparsity, SparseD improves from 29.64% to 43.90% when moving from one-shot masking (f=1.0) to frequent refresh (f=0.2), whereas Step-dLLM improves from 35.18% to 61.79%. This confirms that cross-step mask reuse should be local rather than global.

Head-adaptive budget. The head-adaptive allocation also contributes independently of refresh. Even under the same one-shot setting (f=1.0), Step-dLLM still outperforms SparseD, indicating that a better allocation of sparse budget across heads leads to a better mask even before refresh is introduced. For example, Step-dLLM improves GSM8K from 29.64% to 35.18% at 90% sparsity

Table 3: Ablation on mask refresh frequency (LLaDA-1.5, block size 16). Frequency f means the mask is refreshed every $f \times N_{\text{steps}}$ steps; lower f corresponds to more frequent updates. All values are accuracy (%). **Bold** marks the best per sparsity level.

Sparsity	Metric	SparseD				Step-dLLM			
		f=1.0	f=0.5	f=0.3	f=0.2	f=1.0	f=0.5	f=0.3	f=0.2
95%	GSM8K	9.48	6.29	5.53	6.29	16.83	16.98	17.43	**18.80**
	MATH	12.00	13.80	14.20	13.60	15.40	16.00	15.40	**16.60**
	RULER-VT	86.16	95.40	97.04	96.64	92.32	95.36	96.36	**97.20**
90%	GSM8K	29.64	29.72	34.19	43.90	35.18	40.64	52.24	**61.79**
	MATH	23.00	22.20	23.20	24.80	21.20	22.60	23.60	**26.20**
	RULER-VT	93.64	97.04	97.96	97.96	94.00	96.56	97.04	**97.50**
80%	GSM8K	53.68	64.37	73.09	76.80	57.77	68.23	75.01	**77.56**
	MATH	29.20	31.20	32.40	34.00	31.00	32.00	33.60	**36.40**
	RULER-VT	94.32	–†	–†	–†	98.52	96.64	97.28	**99.88**

† Experiment not conducted for this configuration.

Table 4: Ablation on block size (LLaDA-1.5). Accuracy (%) on short-form benchmarks under block sizes 16 and 32. Step-dLLM consistently outperforms SparseD at both granularities. **Bold** marks the best per row.

Sparsity	Benchmark	Block size = 16		Block size = 32	
		SparseD	Step-dLLM	SparseD	Step-dLLM
95%	GSM8K	9.86	**17.13**	8.64	**19.18**
	MATH	9.40	**13.80**	10.60	**16.20**
	HumanEval	20.12	**21.34**	12.80	**23.78**
90%	GSM8K	29.64	**35.25**	17.51	**39.50**
	MATH	20.00	**21.60**	21.60	**26.00**
	HumanEval	30.49	**32.93**	28.66	**34.15**
80%	GSM8K	53.68	**58.38**	50.87	**63.23**
	MATH	**28.20**	27.60	**31.60**	31.20
	HumanEval	36.59	**37.20**	37.80	**39.12**

and improves RULER-VT from 86.16% to 92.32% at 95% sparsity. This supports the intuition that different heads exhibit different concentration patterns and should not be forced to share a uniform sparsity rule.

Block size. Finally, Table 4 shows that Step-dLLM remains stronger than SparseD at both block sizes 16 and 32, and its advantage becomes larger with coarser blocks. At 90% sparsity on GSM8K, the margin grows from 5.6 points at block size 16 to 22.0 points at block size 32. This trend is expected: when each masking decision removes a larger block, stale masks become more costly, so step-aware refresh and adaptive budget allocation matter even more.

6 Conclusion

We presented **Step-dLLM**, a sparse attention framework designed for the unique dynamics of diffusion language models. Our analysis reveals two properties that prior methods overlook: attention patterns in DLMs are locally consistent but not globally stationary across denoising steps, and different attention heads exhibit markedly different concentration behaviors. Step-dLLM exploits both obser-

vations through three integrated components: cross-step mask reuse with periodic refresh at anchor steps, a head-adaptive budget that allocates sparse computation according to each head's cumulative saliency, and fused Triton kernels that translate algorithmic savings into wall-clock speedups. Extensive experiments on LLaDA-1.5 and Dream-7B-Instruct across mathematical reasoning, code generation, and long-context retrieval benchmarks show that Step-dLLM consistently achieves the best accuracy–sparsity trade-off among sparse baselines, with advantages that grow under high sparsity—where stale masks are most harmful—and on long-context tasks, where the set of salient positions shifts substantially during denoising. At the kernel level, the fused block-sparse attention kernel delivers up to $8.6\times$ speedup over dense Flash Attention at 95% sparsity.

Several directions remain open. The current anchor schedule uses a fixed refresh interval Δ; an adaptive strategy that triggers re-anchoring when the KL divergence between consecutive steps exceeds a threshold could better track abrupt attention transitions while further reducing the number of full-attention computations. Additionally, exploring whether the head-adaptive budget can be extended to full head pruning—entirely skipping heads whose saliency falls below a threshold at certain denoising stages—may yield further efficiency gains. Finally, integrating Step-dLLM with complementary acceleration techniques such as semi-autoregressive block decoding and step-reduction schedules is a promising avenue for compounding speedups in diffusion language model inference. More broadly, the anchor-refresh paradigm—periodically refreshing a sparse mask at selected steps to track shifting attention structure—is not specific to masked discrete diffusion: it may generalize to continuous diffusion transformers, multi-round editing, or any iterative generation process where attention patterns evolve across steps.

References

Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*, 2025.

Jacob Austin, Daniel D Johnson, Jonathan Ho, Daniel Tarlow, and Rianne Van Den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, volume 34, pages 17981–17993, 2021.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, Chengxi Li, Chongxuan Li, Jianguo Li, Zehuan Li, Huabin Liu, Lin Liu, Guoshan Lu, Xiaocheng Lu, Yuxin Ma, Jianfeng Tan, Lanning Wei, Ji-Rong Wen, Yipeng Xing, Xiaolu Zhang, Junbo Zhao, Da Zheng, Jun Zhou, Junlin Zhou, Zhanchao Zhou, Liwang Zhu, and Yihong Zhuang. LLaDA 2.0 technical report. *arXiv preprint arXiv:2512.15745*, 2025. 100B-parameter block diffusion language model.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Yizhao Gao, Zhichen Zeng, Dayou Du, Shijie Cao, Peiyuan Zhou, Jiaxing Qi, Junjie Lai, Hayden Kwok-Hay So, Ting Cao, Fan Yang, and Mao Yang. SeerAttention: Learning intrinsic sparse attention in your LLMs. *arXiv preprint arXiv:2410.13276*, 2024.

Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Minference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning*, 2024.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.

Subham Sekhar Sahoo, Marianne Arriola, Aaron Gokaslan, Edgar Mariano Marroquin, Alexander M Rush, Yair Schiff, Justin T Chiu, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Dor Shmilovich, Tony Wu, Aviad Dahan, and Yuval Domb. LiteAttention: A temporal sparse attention for diffusion transformers. *arXiv preprint arXiv:2511.11062*, 2025.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

Xu Wang, Chenkai Xu, Yijie Jin, Jiachun Jin, Hao Zhang, and Zhijie Deng. Diffusion LLMs can do faster-than-AR inference via discrete diffusion forcing. *arXiv preprint arXiv:2508.09192*, 2025a.

Zeqing Wang, Gongfan Fang, Xinyin Ma, Xingyi Yang, and Xinchao Wang. Sparsed: Sparse attention for diffusion language models. *arXiv preprint arXiv:2509.24014*, 2025b.

Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dLLM: Training-free acceleration for diffusion large language models. *arXiv preprint arXiv:2505.22618*, 2025.

Ruyi Xu, Guangxuan Xiao, Haofeng Huang, Junxian Guo, and Song Han. XAttention: Block sparse attention with antidiagonal scoring. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7B: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025a.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. *arXiv preprint arXiv:2501.01005*, 2025b.

Zebin You, Shen Nie, Xiaolu Zhang, Jun Hu, Jun Zhou, Zhiwu Lu, Ji-Rong Wen, and Chongxuan Li. LLaDA-V: Large language diffusion models for multimodal understanding. *arXiv preprint arXiv:2505.16933*, 2025.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.

Zhihang Yuan, Hanling Zhang, Pu Lu, Xuefei Ning, Linfeng Zhang, Tianchen Zhao, Shengen Yan, Guohao Dai, and Yu Wang. DiTFastAttn: Attention compression for diffusion transformer models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

Jintao Zhang, Haofeng Cheng, Pengle Huang, Jia Wei, Jun Zhu, and Jianfei Chen. SpargeAttn: Accurate sparse attention accelerating any model inference. *arXiv preprint arXiv:2502.18137*, 2025.

Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. LLaDA 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025a.

Fengqi Zhu, Zebin You, Yipeng Xing, Zenan Huang, Lin Liu, Yihong Zhuang, Guoshan Lu, Kangyu Wang, Xudong Wang, Lanning Wei, Hongrui Guo, Jiaqi Hu, Wentao Ye, Tieyuan Chen, Chenchen Li, Chengfu Tang, Haibo Feng, Jun Hu, Jun Zhou, Xiaolu Zhang, Zhenzhong Lan, Junbo Zhao, Da Zheng, Chongxuan Li, Jianguo Li, and Ji-Rong Wen. LLaDA-MoE: A sparse MoE diffusion language model. *arXiv preprint arXiv:2509.24389*, 2025b.

Kan Zhu, Tian Tang, Qinyu Xu, Yile Gu, Zhichen Zeng, Rohan Kadekodi, Liangyu Zhao, Ang Li, Arvind Krishnamurthy, and Baris Kasikci. Tactic: Adaptive sparse attention with clustering and distribution fitting for long-context LLMs. *arXiv preprint arXiv:2502.12216*, 2025c.

A Experimental Details

This appendix provides additional information on models, datasets, and baseline configurations referenced in the main text.

A.1 Model Details

We evaluate Step-dLLM on two publicly available diffusion language models:

- **LLaDA-1.5** [Nie et al., 2025]: A 7B-parameter masked diffusion language model trained on large-scale text corpora. LLaDA uses a Transformer backbone with bidirectional (non-causal) self-attention and generates text by iteratively unmasking tokens over a fixed number of denoising steps.
- **Dream-7B-Instruct** [Ye et al., 2025a]: A 7B-parameter instruction-tuned diffusion language model. Dream follows a similar masked diffusion paradigm but incorporates instruction-following capabilities through supervised fine-tuning.

Both models employ full bidirectional attention at every denoising step, making per-step cost quadratic in the total sequence length. All experiments use the publicly released model weights without any additional fine-tuning.

A.2 Dataset Details

We evaluate on five benchmarks spanning diverse task types:

- **GSM8K**: A dataset of 1,319 grade-school math word problems requiring multi-step arithmetic reasoning. We report exact-match accuracy on the test set.
- **MATH**: A collection of 500 competition-level mathematics problems (MATH-500 subset) covering algebra, geometry, number theory, and more. We report exact-match accuracy.
- **HumanEval**: A benchmark of 164 hand-crafted Python programming problems. We report pass@1 accuracy using greedy decoding.
- **NIAH** (Needle-in-a-Haystack): A long-context retrieval benchmark that embeds target facts ("needles") within distractor text. We evaluate on single-needle (S_1, S_2, S_3), multi-key (MK_1, MK_2, MK_3), multi-query (MQ), and multi-value (MV) subtasks with 4K-token contexts.
- **RULER**: A suite of long-context reasoning tasks including variable tracking (VT), common words extraction (CWE), frequent words extraction (FWE), question answering with short context (QS), and question answering with hard distractors (QH). We use 4K-token contexts.

For all benchmarks, we set the total generation length L_{total} equal to the number of diffusion steps N_{steps} and use greedy decoding (i.e., the most probable token is selected at each unmasking step).

A.3 Baseline Details

Dense. The standard full-attention inference without any sparsification, serving as the upper bound on generation quality.

StreamingLLM. Originally designed for autoregressive models, StreamingLLM retains a fixed set of initial "sink" tokens and a sliding window of recent tokens. We adapt it to the bidirectional dLLM setting by applying the same attention mask at every denoising step. We set the sink token proportion $s_0 = 10\%$ of L_{total} and derive the sliding window size as $W = (\rho - s_0) \cdot L_{\text{total}}$, where ρ is the target sparsity budget.

SparseD [Wang et al., 2025b]. A training-free sparse attention method for dLLMs that computes a block-sparse mask once during the initial denoising steps and reuses it for all subsequent steps. We use the same block size $b = 16$, block length $L = 32$, and sparsity budgets as Step-dLLM. SparseD uses full (dense) attention for the first skip$\% = 5\%$ of denoising steps before switching to sparse attention.

Fast-dLLM [Wu et al., 2025]. A block-wise autoregressive acceleration method for dLLMs. We set the block length $L = 32$ (matching our configuration) and the confidence threshold to 0.9.

A.4 Hardware and Runtime Environment

All experiments are conducted on a single NVIDIA H100 80GB SXM GPU. We use batch size 1 for all models and benchmarks to isolate the effect of sparse attention on per-sample inference latency. The fused Triton kernels (Section 4.3) are compiled with Triton and executed in FP16 precision. Wall-clock times reported in the kernel-level efficiency experiments (Figure 5) are measured as the median of 100 forward passes after 10 warm-up iterations.

A.5 Hyperparameter Summary

Table 5 summarizes the hyperparameter settings used for Step-dLLM across all experiments.

Table 5: Hyperparameter settings for Step-dLLM. Refresh frequency f follows Table 3: we run an anchor (mask refresh) every $f \times N_{\text{steps}}$ denoising steps (rounded to an integer schedule), matching the re-anchor interval Δ in Section 4.1 and Algorithm 1.

Hyperparameter	Short-context tasks (GSM8K, MATH, HumanEval)	Long-context tasks (NIAH, RULER)
Block size (b)	16	16
Block length (L)	32	32
Sparsity budget (ρ)	50%/80%/90%/95%	20%/50%/80%/90%/95%
Refresh frequency (f)	0.2	0.2

B Additional Results

This appendix provides supplementary experimental results omitted from the main text due to space constraints.

Table 6: Long-context results on LLaDA-1.5 with block size 16 (NIAH and RULER benchmarks). All values are accuracy (%). **Bold** marks the best result per sparsity level (excluding Dense).

Method	Sparsity	NIAH								RULER				
		S_1	S_2	S_3	MK_1	MK_2	MK_3	MQ	MV	VT	CWE	FWE	QS	QH
Dense	–	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**52.40**	**64.20**	**91.30**	**76.00**
StreamingLLM	50%	20.80	20.20	18.40	21.20	20.40	15.40	19.20	14.40	30.60	13.40	73.80	81.65	44.40
	80%	5.00	2.40	1.40	3.40	3.60	0.00	2.10	2.10	6.56	3.02	76.27	20.48	22.40
	90%	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	2.12	0.00	0.27	0.00
SparseD	50%	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**99.88**	52.26	**61.40**	**83.17**	77.60
	80%	**100.0**	**100.0**	71.20	**100.0**	**100.0**	74.20	85.85	79.10	94.28	**57.80**	45.40	83.48	77.80
	90%	**100.0**	**100.0**	76.60	**100.0**	99.80	77.20	82.85	80.60	93.32	34.56	42.07	83.13	76.00
	95%	**100.0**	**100.0**	79.40	**100.0**	**100.0**	78.20	81.80	81.10	86.64	39.92	39.00	**81.97**	**73.60**
Step-dLLM	50%	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	99.60	**53.16**	60.87	83.03	**78.60**
	80%	**100.0**	**100.0**	**100.0**	**100.0**	99.80	**99.60**	**99.90**	**98.60**	**98.52**	56.46	**51.73**	**83.73**	**78.20**
	90%	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**97.60**	**99.65**	**95.90**	**94.00**	**54.16**	**46.80**	**83.48**	**76.40**
	95%	**100.0**	**100.0**	**98.80**	**100.0**	**100.0**	**88.40**	**95.25**	**92.30**	**92.32**	**44.76**	**44.07**	81.37	71.40

Table 7: Main results on LLaDA-1.5 with block size 32. All values are accuracy (%). **Bold** marks the best result per sparsity level (excluding Dense).

Method	Sparsity	Short-form			NIAH								RULER				
		GSM8K	Math	HumanE	S_1	S_2	S_3	MK_1	MK_2	MK_3	MQ	MV	VT	CWE	FWE	QS	QH
Dense	–	**81.04**	**37.20**	**40.24**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**52.40**	**64.20**	**91.30**	**76.00**
SparseD	50%	**73.09**	**40.60**	40.85	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	–	–	**100.0**	49.00	**66.93**	**83.43**	77.40
	80%	46.17	**31.60**	37.80	**100.0**	**100.0**	87.60	**100.0**	**100.0**	93.60	–	–	99.56	48.26	52.73	**83.77**	77.00
	90%	17.29	21.60	28.66	**100.0**	**100.0**	88.00	**100.0**	**100.0**	94.00	98.00	90.80	93.32	46.64	49.93	**84.23**	75.40
	95%	12.66	10.60	12.80	**100.0**	**100.0**	**88.20**	**100.0**	**100.0**	92.80	**98.55**	88.80	63.00	43.44	30.80	83.45	73.60
Step-dLLM	50%	**73.09**	40.40	**42.07**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**	–	–	99.92	**49.14**	63.07	82.50	**77.60**
	80%	**57.39**	31.20	**39.12**	**100.0**	**100.0**	**94.80**	**100.0**	**100.0**	**96.80**	–	–	**99.80**	**51.28**	**57.00**	83.23	**77.20**
	90%	**40.86**	**26.00**	**34.15**	**100.0**	**100.0**	**88.20**	**100.0**	**100.0**	**95.40**	**98.60**	**93.65**	**98.48**	**50.02**	**50.13**	82.37	**76.40**
	95%	**21.60**	**16.20**	**23.78**	**100.0**	**100.0**	87.80	**100.0**	**100.0**	**94.00**	98.50	**91.00**	**89.28**	**47.24**	**44.67**	**83.53**	**74.80**

Table 8: Main results on Dream-7B-Instruct with block size 32. All values are accuracy (%). **Bold** marks the best result per sparsity level (excluding Dense).

Method	Sparsity	Short-form			NIAH								RULER				
		GSM8K	Math	HumanE	S_1	S_2	S_3	MK_1	MK_2	MK_3	MQ	MV	VT	CWE	FWE	QS	QH
Dense	–	**77.41**	**45.60**	–†	**100.0**	**100.0**	**91.00**	**98.00**	**99.80**	**77.60**	**97.70**	**96.40**	**96.32**	**87.28**	**79.47**	**79.57**	**64.00**
SparseD	50%	**73.09**	42.00	56.10	**100.0**	99.80	92.40	**98.40**	**99.80**	76.00	**97.55**	**98.05**	96.20	**84.48**	**76.40**	79.73	**64.00**
	80%	46.17	39.20	53.05	**100.0**	**99.80**	75.00	**98.60**	**99.80**	70.60	90.70	97.75	94.12	68.64	72.07	**80.57**	**64.60**
	90%	17.29	29.20	**49.39**	**100.0**	**99.80**	**72.60**	**98.00**	**99.80**	68.80	**89.80**	**96.95**	80.60	53.80	67.87	**79.40**	**64.40**
	95%	12.66	11.60	21.34	**100.0**	**97.80**	**70.20**	**90.20**	99.40	64.20	**78.95**	**81.85**	**24.52**	44.38	**62.60**	77.87	**60.60**
Step-dLLM	50%	**73.09**	**44.60**	**58.54**	**100.0**	**100.0**	**93.20**	**98.40**	**99.80**	**77.20**	96.00	97.95	**96.52**	83.28	76.13	**80.18**	63.80
	80%	**57.39**	**39.80**	**53.66**	**100.0**	**99.80**	**75.60**	98.20	**99.80**	**71.20**	**90.85**	**98.25**	**95.76**	**72.10**	**72.20**	79.17	63.20
	90%	**40.86**	**30.60**	47.56	**100.0**	**99.80**	72.20	94.60	**99.80**	**70.00**	88.15	96.25	**91.56**	**59.36**	**69.60**	78.67	63.00
	95%	**21.60**	**20.40**	**33.54**	**100.0**	86.80	67.40	83.60	**99.60**	**67.60**	74.10	74.40	20.40	**47.07**	59.93	75.77	**60.60**

† HumanEval not evaluated for Dream-7B-Instruct under dense inference.

B.1 LLaDA-1.5 Long-Context Results (Block Size 16)

B.2 LLaDA-1.5 Results with Block Size 32

B.3 Dream-7B-Instruct Results with Block Size 32

B.4 Kernel-Level Efficiency

Table 9: Kernel-level efficiency of Step-dLLM sparse attention. Wall-clock time (ms) for a single attention forward pass at various sequence lengths and sparsity levels. Speedup is computed relative to the dense baseline.

Kernel Size	Sparsity	Baseline (dense)	Ours	Speedup
1K	95%	0.053041	0.01257000	4.22
	90%	0.053041	0.01452900	3.65
	80%	0.053041	0.02579500	2.06
	75%	0.053041	0.03299900	1.61
	50%	0.053041	0.06417000	0.83
2K	95%	0.193364	0.02827213	6.84
	90%	0.193364	0.04997750	3.87
	80%	0.193364	0.09559138	2.02
	75%	0.193364	0.12004263	1.61
	50%	0.193364	0.23373875	0.83
4K	95%	0.763358	0.10975700	6.95
	90%	0.763358	0.20190700	3.78
	80%	0.763358	0.38442200	1.99
	75%	0.763358	0.48581000	1.57
	50%	0.763358	0.61274000	1.25
8K	95%	2.952540	0.36623300	8.06
	90%	2.952540	0.73515000	4.02
	80%	2.952540	1.43898800	2.05
	75%	2.952540	1.83316500	1.61
	50%	2.952540	2.03316500	1.45
	95%	11.837924	1.38469300	8.55

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction claim that Step-dLLM achieves superior accuracy–sparsity trade-offs compared to existing sparse baselines, especially under high sparsity. These claims are supported by extensive experiments on two models (LLaDA-1.5 and Dream-7B-Instruct) across five benchmarks (Tables 1–2) and ablation studies (Tables 3–4). The claimed kernel-level speedups are validated in Figure 5.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 (Conclusion) explicitly discusses several limitations and open directions: the current anchor schedule uses a fixed refresh interval rather than an adaptive one, full head pruning is not yet explored, and integration with complementary acceleration techniques (e.g., semi-autoregressive block decoding) remains future work.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present formal theorems or proofs. The complexity expressions (e.g., $\mathcal{O}(L^2/\Delta + L \cdot k \cdot b)$) follow directly from the algorithm description and do not require formal proof.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 5.1 provides detailed implementation settings (block size, block length, refresh frequency, sparsity budgets, GPU type, batch size). Algorithm 1 gives complete pseudocode. The Appendix provides additional details on models, datasets, baselines, hardware, and a hyperparameter summary table. All benchmarks used are publicly available.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The paper does not currently provide code or supplemental material with reproduction scripts. However, the algorithm is fully described in pseudocode (Algorithm 1), and all benchmarks and base models used are publicly available. We plan to release our code and fused Triton kernels upon acceptance.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5.1 specifies the GPU type, batch size, block size, block length, refresh frequency, and sparsity budgets. The Appendix (Sections A.1–A.5) provides model details, dataset descriptions, baseline configurations, hardware specifications, and a hyperparameter summary table. No training is involved; the method is training-free and applied at inference time only.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The paper does not report error bars or confidence intervals. All experiments use greedy decoding (deterministic), so the primary source of variance is eliminated. However, we acknowledge that variance from different random seeds for the mask schedule or from different input orderings is not characterized.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 5.1 and Appendix A.4 specify the hardware (single NVIDIA H100 80GB SXM GPU), batch size (1), precision (FP16), and the kernel timing methodology (median of 100 forward passes after 10 warm-up iterations).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: This work proposes an inference acceleration technique for existing publicly available language models. It does not involve human subjects, personal data, or deceptive practices, and conforms with the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: This work is a foundational efficiency improvement for diffusion language model inference. It does not introduce new model capabilities or training data, and there is no direct path to specific negative societal applications beyond those already associated with the underlying language models themselves.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: This paper does not release new models, datasets, or pre-trained weights. It proposes an inference-time acceleration method applied to existing publicly available models.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All models (LLaDA-1.5, Dream-7B-Instruct) and benchmarks (GSM8K, MATH, HumanEval, NIAH, RULER) are properly cited. The Triton framework and Flash Attention are also cited. Both base models are publicly released under open licenses.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not release new assets at this time. The fused Triton kernels are described algorithmically in the paper (Section 4.3) but are not yet publicly released.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This work does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: This work does not involve human subjects research.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are the subject of study in this paper (we accelerate diffusion LLM inference), but LLMs are not used as a methodological tool in the research process itself.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.